aidiy_diag 環境確認編
総合テストと操作手順書素材

> 文書: セルフテスト, aidiy_diag, 環境確認編.md | 実装: backend_diag/ui/src/App.vue, backend_diag/ui/src/導入状況/, backend_diag/ui/src/モデル一覧/, backend_diag/ui/src/外部連携/, backend_diag/ui/src/設定モデル/, backend_diag/ui/src/AIモデル/, backend_diag/診断/headless.mjs

総合評価

総合評価：✅ OK (PASS 2件含む)。導入・保存・headless・設定15役割の応答を確認しました。（2026年10月19日・Windows）

プログラムの導入、AIや外部サービスへの接続、結果の保存を調べます。

知りたいこと	現在の結果
AIから実際に返答がある？	✅ OK：今回選んだモデルで、チャット・Liveの挨拶・判断・コード支援の返答を確認しました。すべてのモデルを試した結果ではありません。
Liveはどの条件で通過した？	✅ OK：接続後15秒待って送信し、応答を最大60秒待つ設定で文字の返答を確認しました。この待機はDiagが自動で行います。
音声も確認できた？	✅ OK：追加試験で実音声を取得し、音声認識で「おはようございます」と照合しました。再生APIも正常です。人の耳での聴取とは区別します。

PASS (別途環境調整必要) は2件です。起動していないOllamaとLocalを、それぞれ1件と数えます。今回は設定済み7プロバイダを確認できています。今後キー未登録で試す場合もPASSとし、理由を添えます。設定済みの実接続と、音声の取得・認識照合は通過しました。すべてのモデルの動作保証ではありません。PDF版の総合テスト実施記録も作成しました。Mac／Linuxの実機確認は別途推奨です。

PDF版は本文に合わせて再生成し、全ページの日本語・表・画像・改ページを目視確認しました。

この冊子の進め方

必要なプログラムやAIへの接続を調べ、結果を保存する試験です。導入確認だけならシステム全体を起動する必要はありません。AIの返答を試す5〜6章では、先にシステム起動編でToolsを起動してから始めます。アプリを開く手順はアプリ起動編にあります。

表示・用語	意味
導入状況	必要なプログラムが、このPCに入っているかを調べます。
プロバイダ	AIを提供しているサービスです。
モデル	そのサービスで使うAIの種類です。
外部連携	Discordなど、AiDiy以外のサービスとの接続です。
書込	画面で確認した結果を、ファイルに保存する操作です。
キャッシュ利用	今回新しく取得せず、以前に保存した情報を使った状態です。
headless	ウィンドウを開かず、文字のコマンドで診断する方法です。

✅ **OK** は良好、❌ **NG** は不良、⏸ **未実施** は未確認です。モデル一覧の取得と、AIからの返答は別に試します。

Liveを試すときの待ち方（現在の設定）

Diagが**接続後15秒待って送信**し、**返答を最大60秒待ちます**。再試験も同じです。利用者の追加待機は不要です。

上記の設定で文字の返答を確認し、Live試験は ✅ **OK** です。文字の返答が出たことと、音声を再生して聞けたことは別に記録します。

「テスト中」は結果かエラーが出るまで待ちます。内部の待ち時間は、後半の試験担当者向け記録に掲載しています。

1. 環境確認の画面へ移る

操作手順

1. Diagがミニ表示の場合は、タイトルバーの復帰ボタンを押して通常画面に戻します。
2. 上部の「導入状況」を押します。
3. 画面下部に「診断 接続済み」と表示されていることを確認します。5〜6章を試す場合は「tools オンライン」も確認します。
4. 前回実行日時が表示されている場合は、以前の結果です。今回の試験では、次章の「状況確認」を実行してから結果を読みます。

成功の見分け方と記録すること

「導入状況」が選ばれ、診断項目が表示されれば画面の確認ができます。調べる対象は、Diagを動かしているPCです。Toolsが止まっていても導入確認はできますが、AIの返答を試す「設定モデル」「AIモデル」にはToolsが必要です。

✅ **OK**：通常表示から「導入状況」へ移り、分類・項目・前回実行日時を確認しました。

診断項目が並ぶ入口画面です。この後「状況確認」で今回の結果に更新します。



2. 導入状況を確認して保存する

操作手順

1. 「導入状況」画面で「状況確認」を押します。
2. 確認中の表示が終わり、項目ごとの結果が出るまで待ちます。
3. 左側の分類から、PythonやNode.jsなど、調べたいプログラムの名前を押します。
4. 右側で「何に使うか」「見つかった場所」「バージョン（プログラムの版）」を読みます。必要なら説明やソースの参照箇所も見ます。
5. 「書込」を押し、「書込済」となることを確認します。
6. 保存の試験では、保存ファイルを読み、今回の確認日時と結果が画面に合っていることを確かめます。保存先は下記の記録を参照します。

成功の見分け方と記録すること

右側に選んだ項目の詳細が出れば表示確認は完了です。保存先は `_AIDIY/aidiy_diag/install_info.json`。今回の結果・日時と一致するか比べます。

「未検出」はプログラムが見つからない状態です。使わない追加機能なら全体の故障とは限りません。「検出済み」でも、起動やログインは別に確認します。

✅ **OK（追加試験）**：「状況確認」で19項目すべてOKとなり、Pythonの詳細を表示して「書込」を押しました。Python 3.14.4・検出パス・確認日時と、保存内容が一致しました。

Pythonの詳細と診断済みの項目一覧です。「書込」により今回の結果を保存しました。



3. モデル一覧を取得して保存する

操作手順

1. 上部の「モデル一覧」を押します。
2. 調べたいAIサービス（プロバイダ）の行を選び、設定済みか、何を調べる試験かを読みます。
3. 1件だけなら行の「テスト」、まとめて取得するなら「一覧再取得」を押します。
4. 取得中の表示が終わるまで待ち、モデル名・確認日時・説明を読みます。
5. 「書込」を押し、「書込済」となることを確認します。保存の試験では、保存内容と画面のモデル名・日時を比べます。

「キャッシュ利用」は以前の一覧を使った状態です。今回インターネットから新しく取得できた場合とは分けて記録します。

成功の見分け方と記録すること

選んだサービスの結果と確認日時が更新され、モデル情報を `_AIDIY/aidiy_diag/` へ保存できれば、この章の確認ができます。取得できなかった場合は、設定がないのか、通信に失敗したのか、表示された理由を記録します。

モデル名が出た場合でも、公開情報やあらかじめ用意された候補を表示しただけの場合があります。接続に使うキーが有効か、実際に文章や音声を作れるかは、5〜6章で別に試します。

✅ **OK（取得・保存）**：Geminiの個別テストと「一覧再取得」を実施し、9プロバイダの結果を書き込みました。設定済み7プロバイダはOK、backend_localは固定候補を示すPASSです。Ollamaはサーバー停止中のため、画面と保存内容にはNGが残っています。今回の総合テストでは環境条件としてPASS (別途環境調整必要)に分類します。Localも未起動なので同じ分類です。いずれも会話成功とは判定しません。

一括取得と保存後のモデル一覧です。Geminiのモデル名2件と確認日時が保存内容に一致しました。



4. 外部連携を確認して保存する

操作手順

1. 上部の「外部連携」を押します。
2. Hugging FaceやDiscordなど、調べたいサービスの行を選びます。設定の有無と、何を確認するかの説明を読みます。
3. 1件だけなら行の「テスト」、まとめて調べるなら「状況確認」を押します。
4. 結果が出るまで待ち、結果・説明・確認日時を確認します。未設定なら、その理由も記録します。
5. 「書込」を押し、保存した結果と画面の内容を比べます。

この操作は接続に使う情報が有効かを調べます。Discordのチャンネルへメッセージを送る試験は、この章には含みません。

成功の見分け方と記録すること

設定済みのサービスを調べ、接続の確認結果が表示されることを確かめます。未設定の行は、試せない理由と一緒に**PASS（別途環境調整必要）**と記録します。保存先は `_AIDIY/aidiy_diag/integration_info.json` です。

Discordでは、Bot自身の情報が取得できるかを調べます。メッセージ投稿や音声会話まで成功したという結果ではありません。Hugging Faceでも、使う権限の確認と、ファイルを実際に書き込む操作は別に扱います。

✅ **OK（認証確認・保存）**：Hugging Face読取キーの個別テストと、外部連携の「状況確認」を実施しました。読取・書込キー、Discord Botの3件すべてOKで、「書込」後の保存内容と結果・確認日時が一致しました。投稿やリポジトリへの書込は試験範囲に含みません。

3件の認証確認が終わった画面です。表示は認証と権限の確認結果です。



5. 設定中のモデルで応答を確認する

操作手順

1. 画面下部の「tools オンライン」を確認し、上部の「設定モデル」を押します。押せない場合はToolsの起動状態を確認します。
2. パネル見出しの「再読込」を押し、現在設定されているAIを読み直します。
3. 左の一覧から、今回試す役割を選びます。CHATは文字会話、LIVEは音声会話、DECISIONは選択や判断、CODEはコード支援です。プロバイダとモデル名も控えます。

4. 1行だけ試す場合は、その行の「テスト」を押します。設定中のモデルを順番に試す場合は、パネル内の「動作確認」を押します。
5. テスト中の表示が終わるまで待ち、右側の「応答」にAIの返答が出たことを確認します。途中でやめる場合は「中止」を押します。
6. Liveの音声プレーヤーが出た場合は、再生ボタンを押して、再生位置が進むか確認します。実際に聞こえた内容は、文字の返答とは別に記録します。

画面の一番右上の「全テスト」は、AIの返答を試すボタンではありません。役割の違いは7章の表で確認できます。

成功の見分け方と記録すること

「応答」にAIの返答が出たことが成功の目印です。「テスト中」や「未設定」だけでは合格にしません。「共有元」がある行は、同じ接続条件で行った別の行の結果を使っています。その表示も記録します。AIサービスの利用条件によっては、試験にも料金がかかります。

最終結果と画面

✅ **OK**：設定モデル15役割すべての応答を確認しました。TASK／TEAMの6役割は、同じ設定のCODE1結果を共有します。Live実行中の「中止」で試行を解除し、再実行できることも確認しました。

全15役割が正常応答となった画面です。



6. AIモデルと入力を選んで試す

操作手順

1. 「tools オンライン」を確認して、上部の「AIモデル」を押します。
2. パネル内の「候補を再読込」を押します。
3. CHAT・LIVE・DECISION・CODEのタブを1つずつ選び、使うプロバイダとモデルを確認します。LIVEでは音声の種類、DECISIONでは質問文や選択肢も確認します。
4. 会話の入力欄には、次の文を使います。

> おはよう。稼働試験です。挨拶を一文だけ返してください。ファイル変更やコマンド実行はしないでください。

5. 各タブの入力を用意したら、パネル内の「動作確認」または画面右上の「一括試行」を1回押します。CHAT → LIVE → DECISION → CODEの順に進みます。
6. 完了するとCHATへ戻ります。他の3つのタブも押し直し、結果と返答を確認します。AIが返した文章や判断が表示されていれば、返答の確認ができます。
7. 選んだモデル名、入力、実際の返答、結果を記録します。この画面でのモデル選択は、本体の設定を書き換えません。

成功の見分け方と記録すること

選んだモデルと、実際に返したモデルの情報が表示される場合は、両方を記録します。「auto」は自動選択という意味なので、結果に出たモデル名も確認します。文字の返答・判断結果と、音声の取得・再生を分けて記録できれば、この章の結果を読み比べられます。

live_checkの最終確認（2026年10月19日）

✅ **OK**：`backend_tools/tools_proc/live_model_check.py` は、接続準備完了を待った後に15秒待機し、送信後の応答を最大60秒待つ実装でした。条件に合っているため、今回の実装修正は不要でした。

関連の `test_live_model_check.py` は22件すべて成功しました。さらに起動中の `/internal/live_model_check` へ直接HTTPで2回試行し、それぞれ新しいLive接続の初回送信で `status: OK`、`reason: response_received` と「おはようございます!」の返答を得ました。全処理時間（接続と15秒待機を含む）は34.15秒・21.13秒でした。2回目は1.24秒の音声も取得しました。これはAPIと自動テストの追加確認で、Diag画面の操作を今回再実施した記録ではありません。

※注意:15s待機後送信しました。

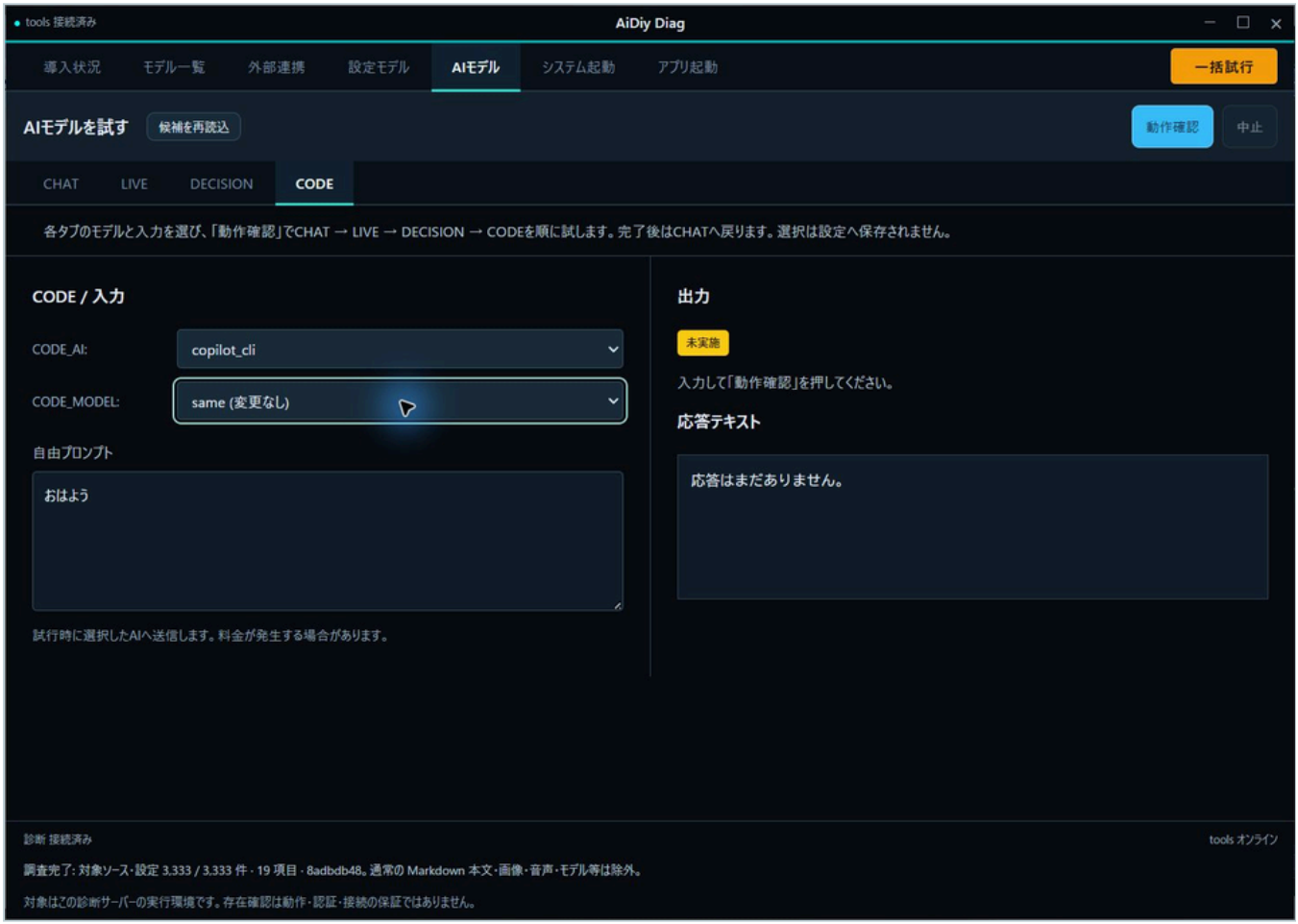
Providerは `freeai_live`、モデルは `gemini-2.5-flash-native-audio-preview-09-2025`、声は `Zephyr` です。本体設定は変更していません。

今回の最終結果：接続後15秒・応答待ち最大60秒

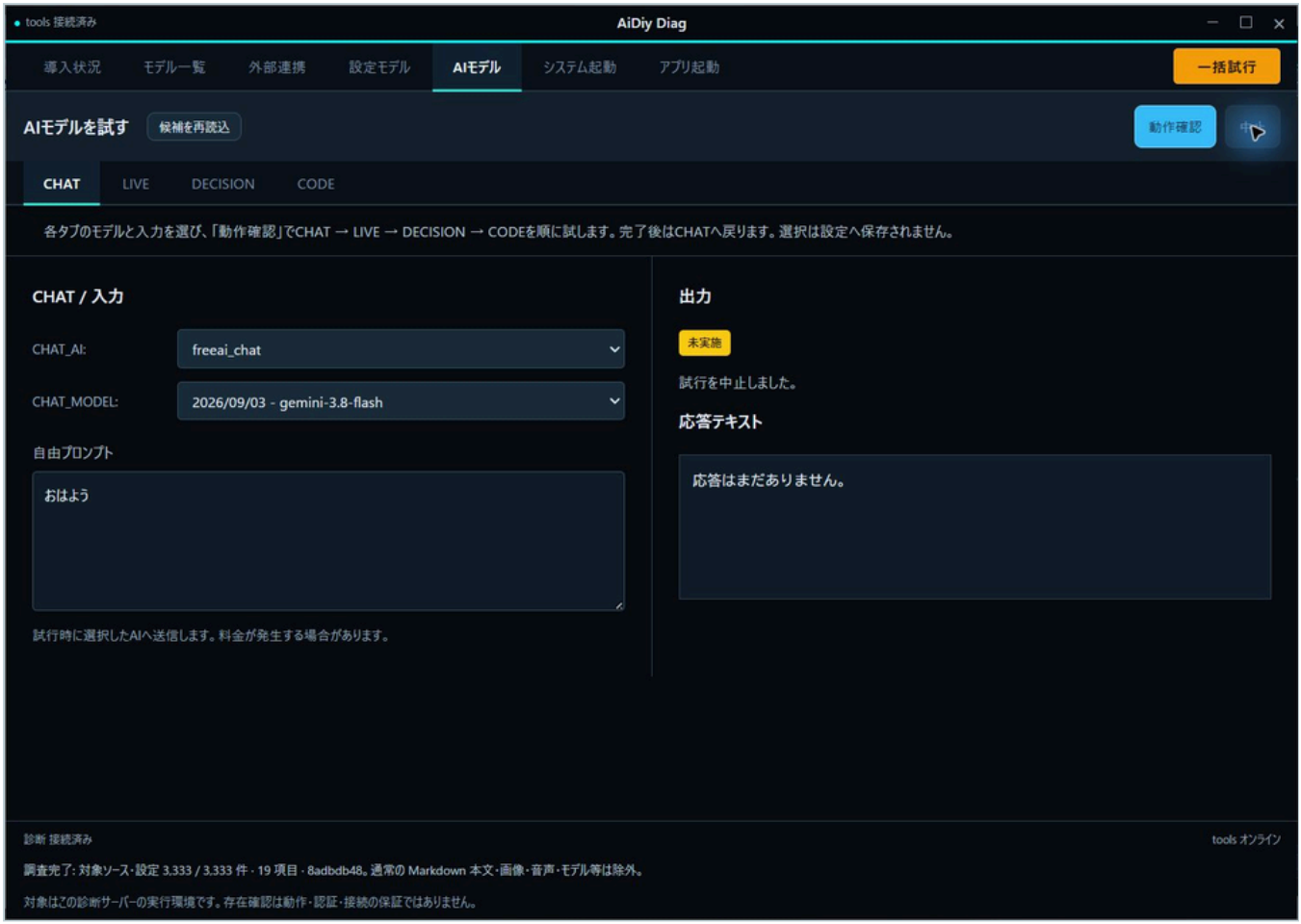
✅ **OK（追加試験）**：「候補を再読込」後に候補が再表示され、CODEを `copilot_cli / same` へ変更できました。「動作確認」中の「中止」で実行を解除し、再び「動作確認」を押すと4種類の試行が終わり、CHATへ戻りました。本体のモデル設定は変更していません。

種類	選択した接続・モデル	今回の結果
CHAT	freeai_chat / gemini-3.8-flash	✅ OK：「おはようございます。」から始まる実回答
LIVE	freeai_live / gemini-2.5-flash-native-audio-preview-09-2025 / Zephyr	✅ OK：「おはようございます。調子はいかがですか?」と返答。2.80秒の音声取得と再生位置の終端到達も確認
DECISION	auto / auto → openai_oauth/gpt-6-luna	✅ OK：挨拶の丁寧さをlowと判断。confidenceは0.95
CODE	copilot_cli / same	✅ OK：「おはようございます！」から始まる実回答。実行モデル名は応答に含まれず、推測していません。

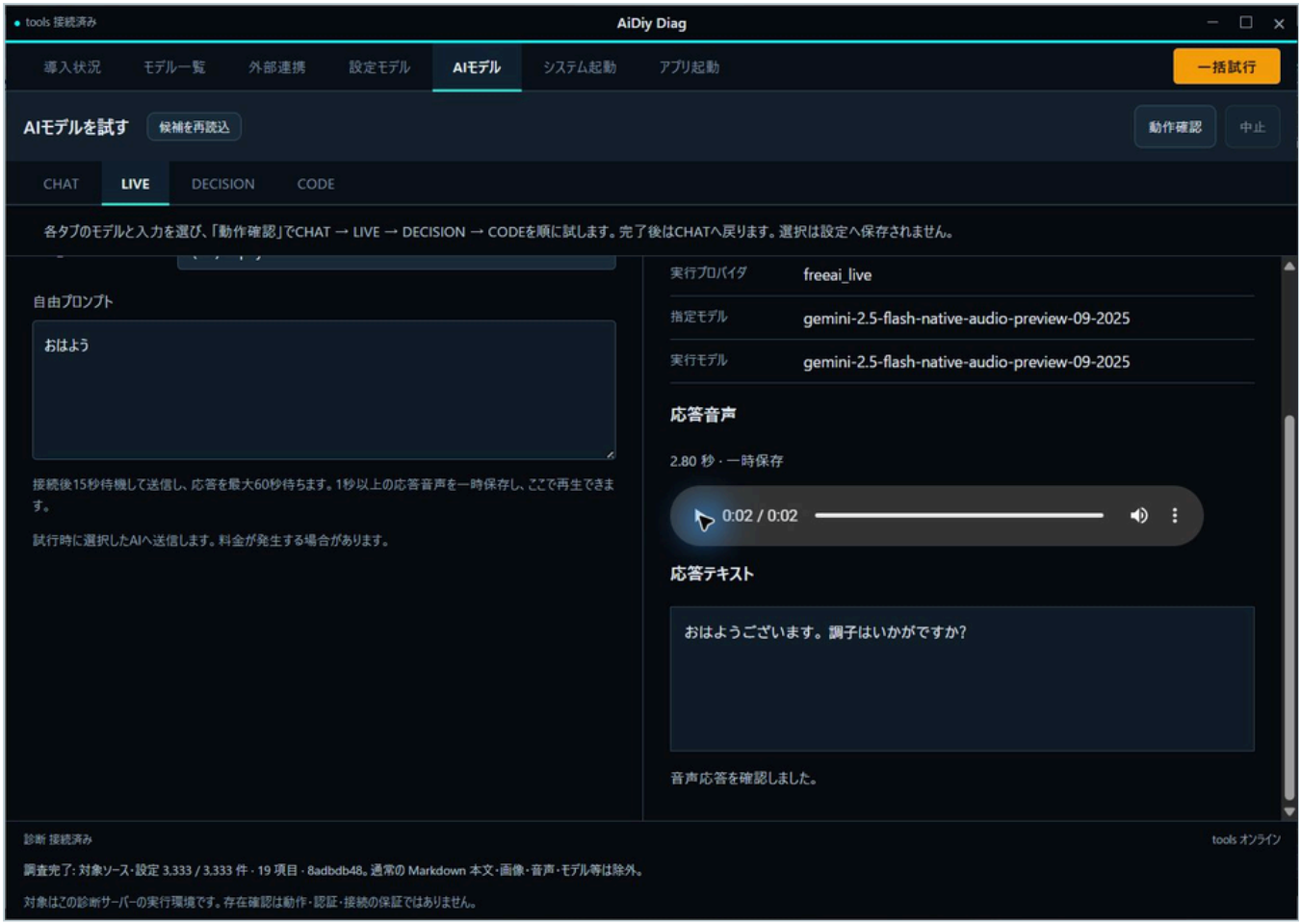
モデル候補を切り替えた画面です。`copilot_cli` と `same`（**変更なし**） を選択できました。



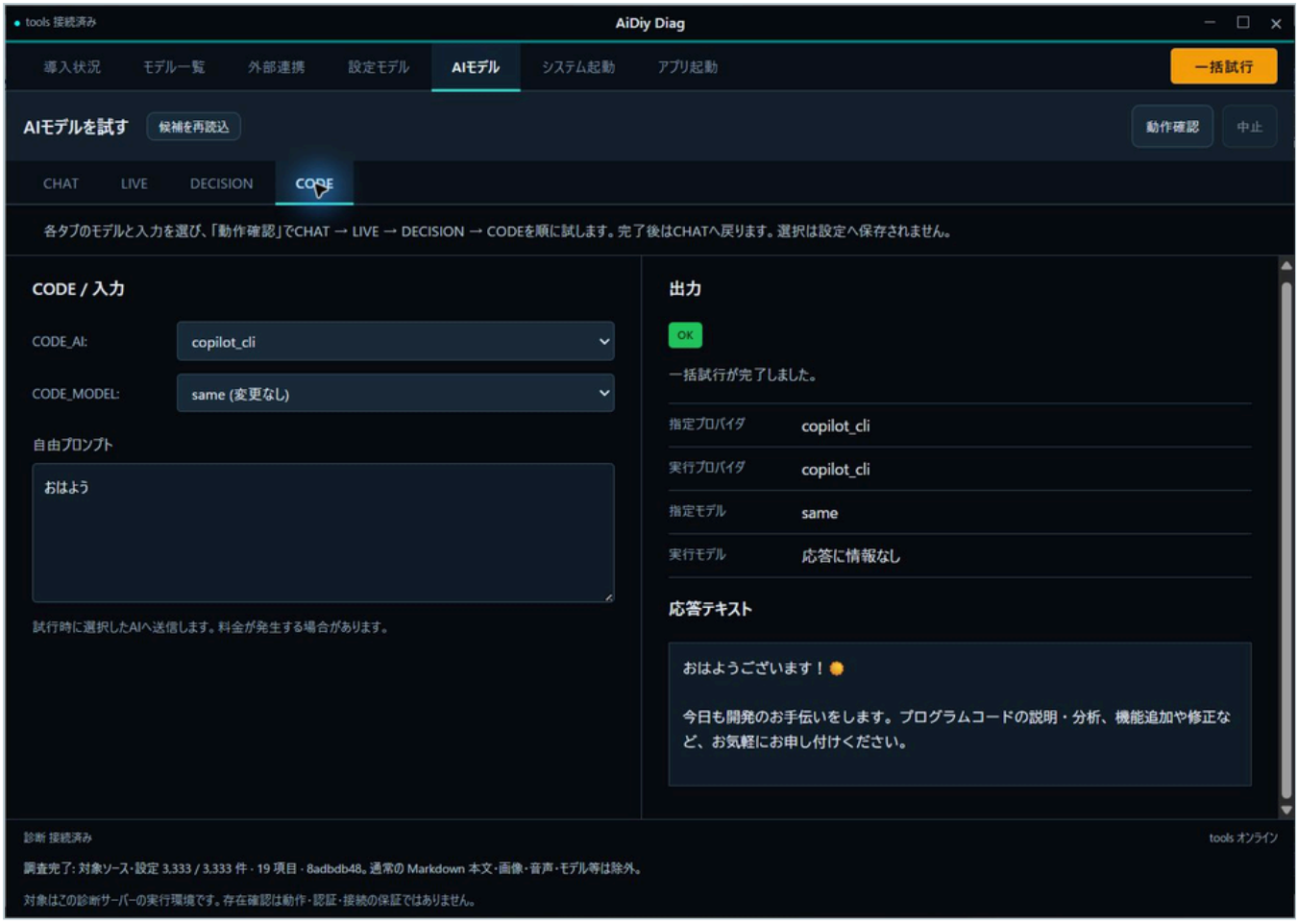
途中で「中止」を押した結果です。「試行を中止しました。」となり、「動作確認」を再び押せる状態に戻りました。



再実行後のLiveです。15秒・60秒の説明、文字の返答、2.80秒の音声と再生完了位置を確認しました。音声内容の聴取は未実施です。



CODEの結果です。`copilot_cli / same` の挨拶への返答が表示されています。



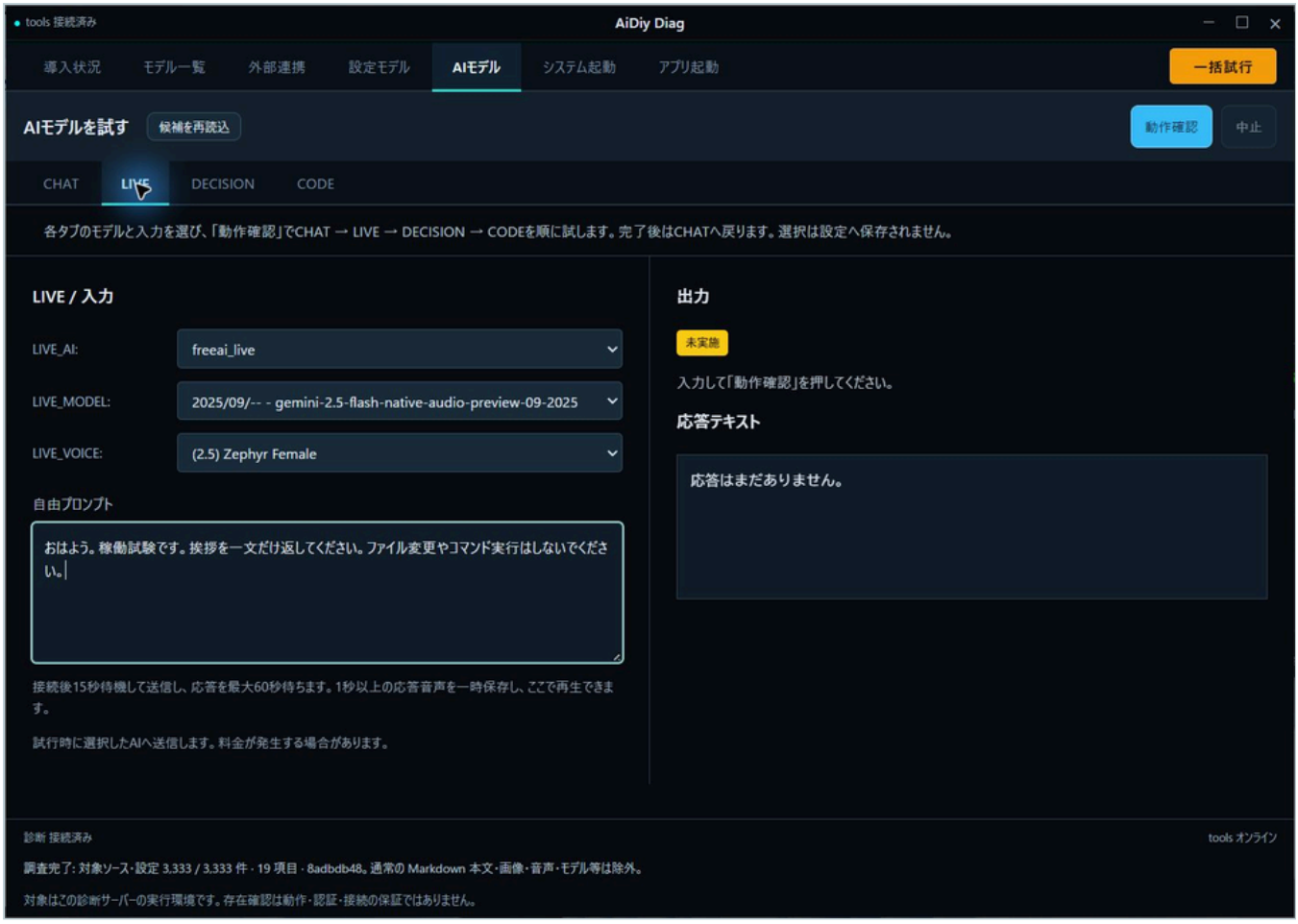
再読込・CHAT・判断の追加画面も補助資料として保持しています。各操作の判定は本章に記載した結果のとおりです。

同じ待機設定で行った長い試験文の確認

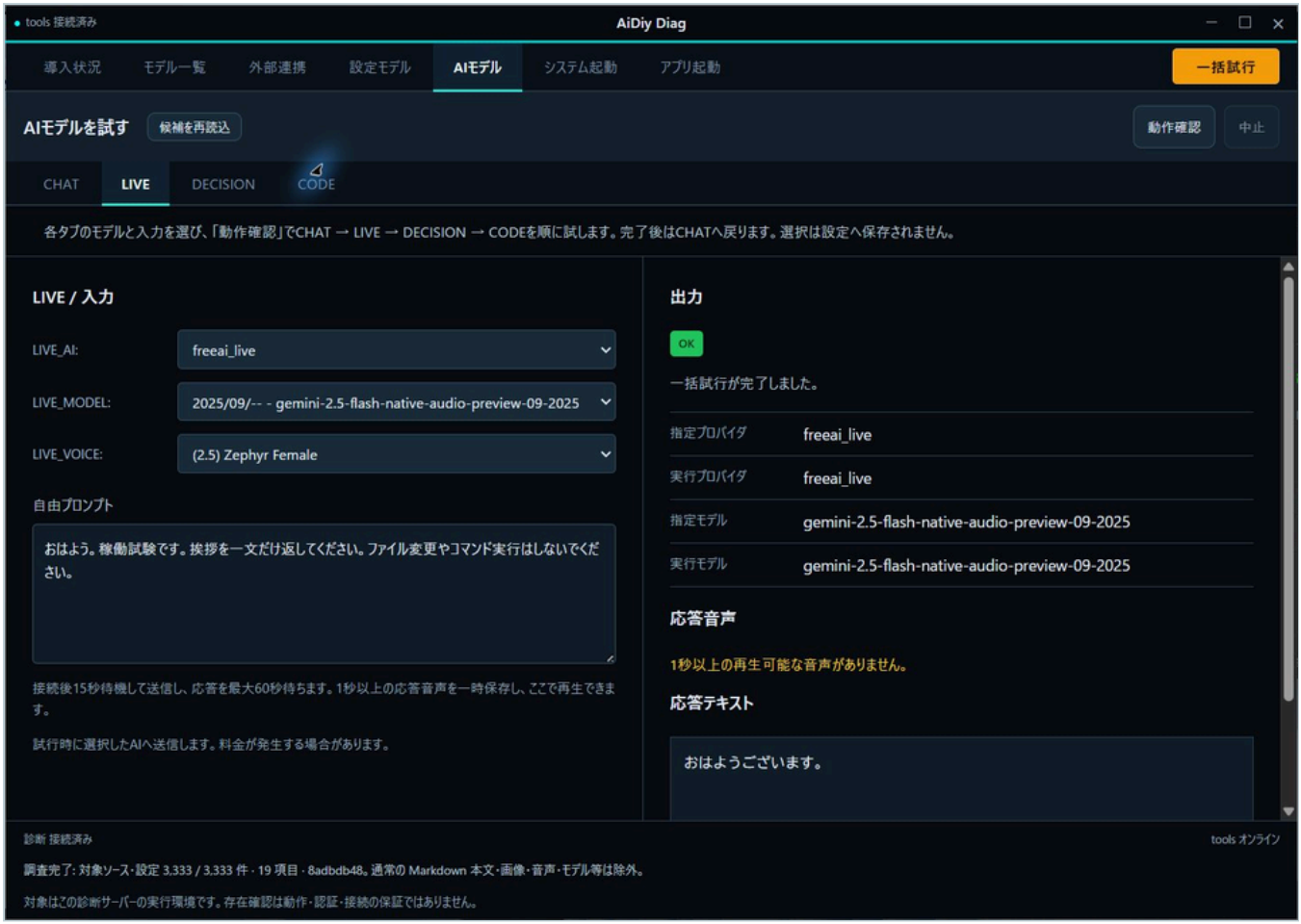
✅ **OK**：接続後15秒・応答待ち最大60秒の条件で、モデル・声・試験文を設定し、「動作確認」を押しました。内部処理を確認する自動テスト51件と、配布する画面への変更反映も確認済みです。

AIから「おはようございます。」と返答があり、画面の結果もOKでした。今回のモデルは `freeai_live / gemini-2.5-flash-native-audio-preview-09-2025` です。現在の15秒・最大60秒の設定で、文字の返答の試験は通過しました。今回は保存できる長さの音声がないという表示だったため、音声再生まで成功した結果にはしていません。

現在の15秒・最大60秒の設定で、前回と同じ試験文を用意した画面です。



現在の設定での最終結果です。LiveがOKとなり、「おはようございます。」と返答がありました。



接続後15秒・応答待ち最大60秒の条件で、文字の返答を確認しました。この回は再生できる長さの音声がなく、音声再生の成功とは判定していません。

試験担当者向けの補足

接続準備は最大20秒、接続準備・待機・終了処理を含む内部の通信は最大135秒、1回の試験全体は最大150秒にしています。画面にmsと出る場合はミリ秒で、1000 msが1秒です。

追加確認：実音声の内容と再生

✅ **OK**：接続後15秒・応答待ち最大60秒で、1.32秒・24kHz・モノラルの音声を取得しました。別の音声認識で「おはようございます」となり、実際の返答文と一致しました。Windowsの再生APIも正常終了しました。人の耳での聴取・スピーカーからの再録音は行っていません。

1.32秒の実音声WAVを保持しています。音声認識は返答文と一致し、Windowsの再生処理も正常終了しました。人の耳での聴取は未実施です。

7. 全テストとheadlessを使い分ける

操作手順

同じ「まとめて実行」のように見えても、ボタンによって調べるものが違います。目的に合う行を選んでください。

やりたいこと	使う場所・操作	完了したときに確認するもの
必要なプログラム・モデル一覧・外部接続をまとめて調べ、結果を保存する	Diagの通常画面で右上の「全テスト」	各結果と保存が完了し、最後に「導入状況」へ戻ったことを確認します。AIへの挨拶送信やシステム起動は含みません。
AIに実際に入力して返答を試す	「AIモデル」の右上「一括試行」	CHAT・LIVE・DECISION・CODEの各タブで返答や判断結果を確認します。
アプリに必要なシステムを起動する	「システム起動」の右上「設定起動」	選んだ機能の「稼働中」と起動完了を確認します。詳しくはシステム起動編へ進みます。
画面を開かず診断結果だけ保存する	ターミナルで `aidiy_diag --headless` を実行	コマンドの終了結果と、保存された導入状況・モデル情報を確認します。外部連携の認証確認、AIの返答、システム起動は含みません。

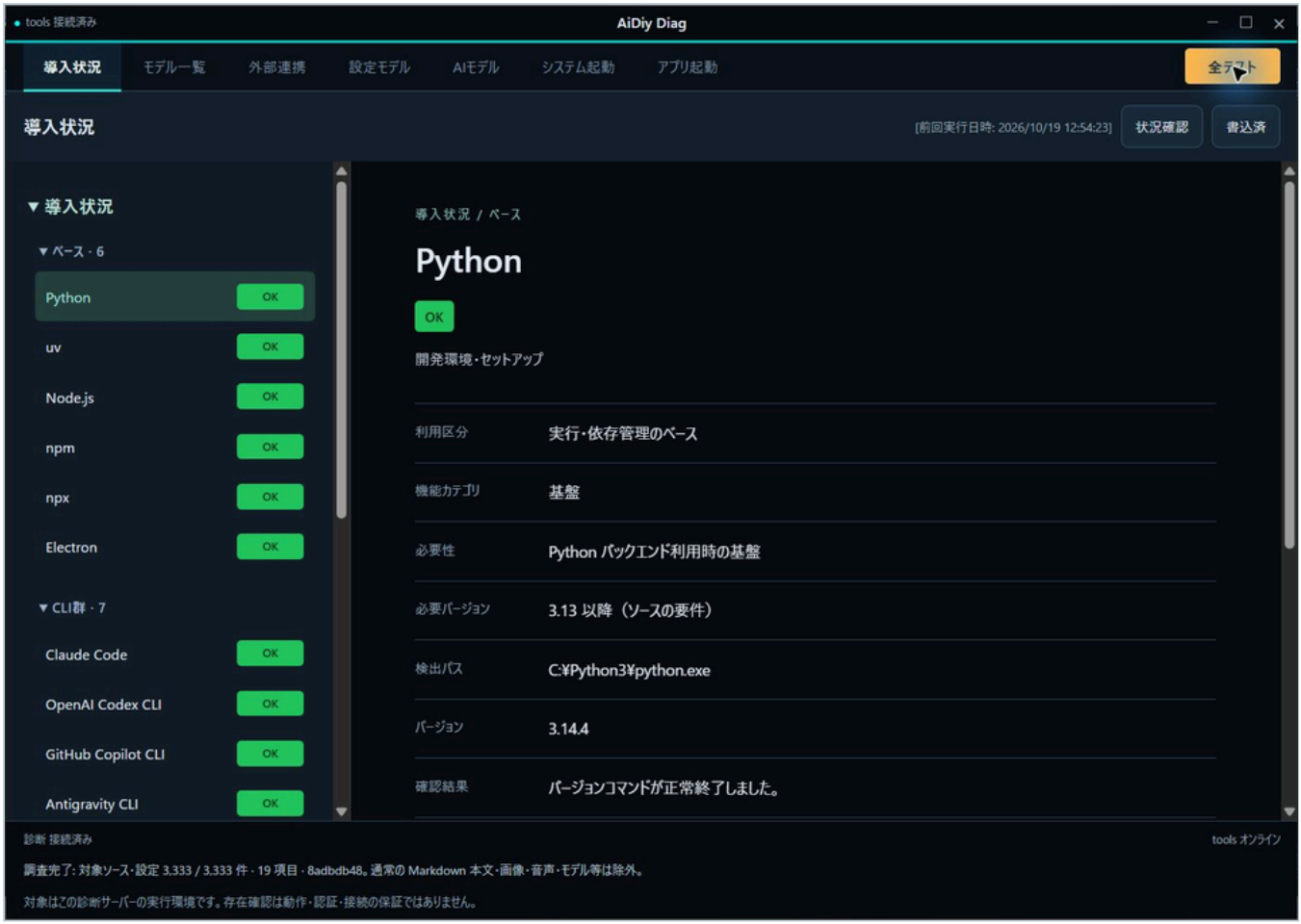
画面を開かない場合でも、モデル一覧の取得にはサービスへの通信が発生します。headlessの試験結果は、通常画面での試験と分けて記録します。

成功の見分け方と記録すること

「全テスト」は各段階の結果・書込・最後の表示タブを確認します。headlessは終了コード・出力・生成ファイルを照合します。終了コード0だけで、任意プログラムの未検出や全モデルの生成可否まで合格にしません。失敗した保存処理については前回結果を壊さないことも確認します。

✅ **OK（追加試験）**：画面右上の「全テスト」で、導入確認・保存、モデル一覧の再取得・保存、外部連携の確認・保存まで完了し、導入状況へ戻りました。3種類の保存ファイルが今回の日時に更新されていることも確認しました。

全テスト完了後の画面です。導入状況は19項目すべてOKで、書込済みです。



続いてコマンドからheadlessを実行し、終了コード0を確認しました。画面とheadlessの導入結果19件は、項目名・状態・パス・バージョンが完全一致しました。headlessは外部連携の認証確認を行いません。

8. 試験担当者向け：次回の記録方法

画像の名前・保存先・自動試験の記録方法を開く

本書1〜7章を順に実施します。Computer Useは画面の観察 → 1操作 → 再観察で進め、確認できた結果だけを判定します。システム起動編6章の記録方法と `scripts/checks/実画面試験記録.mjs` を共用する。単体テストや合成画面と、実環境の実画面試験を分けます。

保存先は `_AIDIY/knowledge/images/セルフテスト,aidiy_diag,環境確認編/` です。新しい画像名は、下の例の前に `<実施ID>__` を付けます。章番号を先頭1桁とし、章内で01から連番にします。既存画像は上書きせず、別冊内の番号とは独立させます。

章	ファイル名の例	残す画面
1	aidiy_diag-環境確認_101_導入状況表示画面.png	環境確認への切替と接続状態
2	aidiy_diag-環境確認_201_導入状況確認完了画面.png	結果・選択項目の詳細・書込済
3	aidiy_diag-環境確認_301_モデル一覧取得完了画面.png	取得結果・日時・保存状態
4	aidiy_diag-環境確認_401_外部連携確認完了画面.png	認証確認の結果・説明
5	aidiy_diag-環境確認_501_設定モデル応答画面.png	設定中モデルの実回答
6	aidiy_diag-環境確認_601_AIモデル応答画面.png	選択したモデル・入力・実回答
7	aidiy_diag-環境確認_701_全テスト完了画面.png	全処理後の導入状況表示

各章の「未実施」を、実施日時・操作・期待結果・実際の結果・判定と、説明付き画像へ置き換えます。課題を解消した場合は、結果と画像を最終状態へ差し替えます。解消済みの経緯は記載しません。保存内容との照合やheadlessの結果は、比較した項目と一致・不一致を本文の表にまとめます。認証キーや秘密値を証跡へ含めません。

2026年10月19日に1〜7章を実施しました。設定モデルは15役割、AIモデルは4種類の返答を確認しました。Liveは接続後15秒・応答待ち最大60秒で通過しました。6章のAIモデル試験では2.80秒の音声再生を確認し、同章の追加試験では1.32秒の音声を取得して、音声認識で返答文との一致を確認しました。人の耳での聴取は未実施です。上の画像名の表は次回の採番例で、取得済み画像と結果は各章の本文を参照します。